Lab 3E: From Dashboard to Production (Optional)

Duration: 45 minutes
Difficulty: Advanced
Continues from: Labs 3A-3D (dashboard project with commands, hooks, MCP, subagents)

Overview

This optional lab ties everything together. You'll use every tool you've built this week — CLAUDE.md, custom commands, hooks, MCP, and subagents — to transform your dashboard from a lab project into a production-ready system. This is the capstone exercise.

What You'll Build

- Production error handling and logging
- Environment-based configuration (dev/staging/production)
- API rate limiting and input sanitization
- Automated documentation generation
- A full security audit using your subagent
- Deployment-ready configuration

Setup

Continue in your dashboard project:

```
cd business-dashboard
claude
```

Clear Context

```
/clear
```

Phase 1: Production Error Handling

Your dashboard works, but errors return raw stack traces. Fix that:

```
Refactor all error handling across the project. Every endpoint should:
1. Catch all errors with try/catch
2. Log the full error to console.error (for server logs)
3. Return a user-friendly error response: { error: "message", code: "ERROR_CODE" }
4. NEVER expose stack traces, file paths, or internal details in the response
5. Use appropriate HTTP status codes (400 for bad input, 404 for not found, 500 for
server errors)

Apply this to every route file in src/routes/
```

Verify with your review command:

```
/review src/routes/
```

Phase 2: Environment Configuration

Hardcoded values don't belong in production code:

```
Create a src/config.js that reads from environment variables with sensible defaults:
- PORT (default: 3000)
- NODE_ENV (default: development)
- DATABASE_URL (default: sqlite://./data/dashboard.db)
- LOG_LEVEL (default: info in production, debug in development)
- RATE_LIMIT_WINDOW (default: 15 minutes)
- RATE_LIMIT_MAX (default: 100 requests per window)

Create a .env.example showing all variables with placeholder values.
Update server.js to use config.js instead of hardcoded values.
Add .env to .gitignore if it isn't already.
```

Phase 3: Rate Limiting and Input Sanitization

Protect your API from abuse:

```
Add rate limiting middleware using express-rate-limit.
Apply a global rate limit using the values from config.js.
Add a stricter rate limit (10 requests per minute) to any POST/PUT/DELETE endpoints.

Then add input sanitization to all endpoints that accept user input.
Validate request bodies, query parameters, and URL parameters.
Reject invalid input with a 400 status code and descriptive error message.
```

Test the rate limiting:

```
Write a quick test that sends 15 rapid requests to POST /api/tasks
and verifies that rate limiting kicks in
```

Phase 4: Security Audit

Use your security auditor subagent from Lab 3D:

```
@security-auditor Perform a complete security audit of this project.
Check for: dependency vulnerabilities, hardcoded secrets, injection risks,
missing input validation, improper error handling, and CORS configuration.
```

Review the findings. Fix any CRITICAL issues:

```
Fix all CRITICAL security issues from the audit.
Run the tests after each fix.
```

Run the audit again to verify:

```
@security-auditor Re-audit. Have all CRITICAL issues been resolved?
```

Phase 5: Generate Documentation

Use your doc command and doc-writer subagent:

```
/doc src/routes/
```

Save the output:

```
Save the complete API documentation to docs/API.md
```

Now generate a README:

```
@doc-writer Generate a production README.md for this project. Include:
- Project description
- Quick start guide (prerequisites, install, run)
- Environment variables reference (from .env.example)
- API endpoints summary with links to full docs
- Development setup
- Testing instructions
- Deployment notes
```

Phase 6: Deploy Check

Run your deploy-check command as the final gate:

```
/deploy-check
```

Fix any remaining failures. When you get a clean GO verdict:

```
Create a final commit with message "feat: production-ready dashboard with
error handling, rate limiting, input validation, and documentation"
```

What You Built This Week

Day	What	Tools Used
Day 2	Dashboard application	Agentic loop, CLAUDE.md, @ mentions
Lab 3A	Team workflow commands	Custom commands
Lab 3B	Safety guardrails	Hooks
Lab 3C	External tool integration	MCP servers
Lab 3D	Specialized AI workers	Subagents
Lab 3E	Production-ready system	Everything combined

You started with a single prompt on Day 2 and ended with a production-grade application, complete with safety guardrails, automated documentation, and a team of specialized AI assistants.

This is AI-assisted development at scale.

Congratulations — you've completed the full Claude Code training.